ECM2414 Continuous Assessment

50:50

Development Log

Date	Time	Duration	Sam Role	Charles Role	Sam Signature	Charles Signature
8/11/23	18:00	2 Hours	Driver	Observer	096112	096195
12/11/23	10:00	2 Hours	Observer	Driver	096112	096195
13/11/23	15:30	2 Hours	Driver	Observer	096112	096195
15/11/23	16:45	1 Hour	Driver	Observer	096112	096195
23/11/23	16:45	1 Hour 15 Minutes	Observer	Driver	096112	096195
27/11/23	16:00	2 Hours	Observer	Driver	096112	096195
29/11/23	16:00	2 Hours	Driver	Observer	096112	096195

Code Design and Performance

CardGame

The CardGame class is the main game class, containing the main function used for running the code. It has 2 private static attributes which are ArrayLists storing the players and decks inside the current game. These attributes have public getter methods, and players has a setter method used exclusively for testing purposes.

The main method is used to start the game. It takes 2 inputs, the number of players which is checked to be a non-negative integer before preceding, and a file path for the pack file to be used. The loadPack function is used to read this file, returning an empty ArrayList if the pack doesn't exist or is invalid, such as containing integers <= 0 or being the wrong size for the number of players chosen. Once both inputs have been taken and verified, the main function will call private methods inside CardGame, starting with createDecks and addPlayers, which fill the private players and decks attributes, assigning each player their left and right deck on creation. 4 cards are then dealt to each player using the dealCardsToPlayers function, and the dealCardsToDeck does the same with the remaining cards in the pack. Once everything is set, each player thread is started.

Once the game is won, the static method gameOver will be called by the winning thread, passing through its identifier to indicate it is the winner. This can then be displayed in the console. The main method can then interrupt all other player threads to make sure they finish up. This prevents any issues where players are waiting for decks to have cards available to take, as it will wake them up and cause them to start checking if the game is over again, at which point they should clean themselves up and exit. The method then calls each deck's outputDeck function to create the output files for the deck.

Card

Card is a simple class representing a card in the game and holding a private integer value, the face value of the card, which is set at object creation. This attribute has a getter method getValue(), and the toString() method returns the value of the card as a string to be used in the toString() method of the CardCollection class.

CardCollection

Card Collection is created as an abstract model to be used by both the Deck and Hand since they rely on similar underlying methods. It implements a LinkedBlockingQueue, which is part of the java.util.concurrent library. This functions similarly to a normal queue, however is built to be thread safe, with waiting built-in so threads wait until the queue contains a value before taking anything out. These methods can also be interrupted like the standard wait() method for synchronisation. This blocking queue is protected so that it can be used inside concrete methods for the Deck and Hand classes.

It implements fundamental methods to get, add and remove cards from the queue. The size of the queue can also be accessed, along with getting an iterator object that can be used to iterate through the queue inside functions outside Hand and Deck, for example, in the checkWin() method of the Player class. The toString method is also overwritten for this class, outputting the value of the cards in the queue, separated by a space. This toString method is used by both the decks and hands.

Deck

Deck extends the abstract CardCollection method, adding a private identifier to represent which deck the object refers to, e.g: Deck 1. This identifier is generated automatically at object instantiation using a static AtomicInteger counter. The only additional methods added by deck are a getter method for the identifier (getIdentifier()), and an outputDeck() function which uses a buffered FileWriter to output the contents of the deck, using the classes toString() method to simplify the process. This method is designed to attempt to write to the file 3 times should an IOException occur, ensuring the output is consistent at the end of each game.

Hand

Hand is a simpler extension of CardCollection compared to deck. It has a second removeCard function allowing a Card object to be removed from anywhere in the queue, provided that queue contains the Card. It also implements a getRandomCard() function that returns a random card from inside the queue which can be used when deciding what card to discard during the round. This uses the built-in Java.util.Random module for generating a random integer.

Player

The Player class represents the threads within the application. It extends the Thread class, and overrides the built-in run() method with the game play, such that it initially checks the hand it's been dealt to see if it wins, before continually taking a card from the deck to it's "left", i.e. the deck with the same identifier as it, and outputting to the deck to the right (deck with an identifier 1 greater than the players identifier or 1 if it has the largest player identifier in the game), completing the circular setup required for the card game. Each gameplay turn is done as an atomic action, only starting if the game is not over, and checking if the player has won at the end of each turn. This ensures all players always finish the game with 4 cards in their hand. drawCard() can be used to place a card into a players hand during the start of the game.

Similarly to Deck, the Player class implements an auto generated unique identifier using a static AtomicInteger counter so that it's thread safe. Each player has a unique instance of the Hand class, stored a private attribute with a getter method getHand() used for testing only. The "left" and "right" decks are passed in when a new player is instantiated and stored as private attributes, similarly with getter methods for testing purposes. There is also a public getter for the identifier attribute. Other methods, such as startOutput() and writeOutput() are used for handling the output file stream, which is stored as a private attribute so that it can be accessed by any method.

To track the winner of the game, the Player class implements a second static AtomicInteger called winner. This is initialised as -1 to indicate no winner and set to the identifier of the winning player once a player thread decides it has a winning hand. The static method checkGameOver() returns a boolean output depending on if the winner is still -1 or not. When a player wins, it calls the gameOver() method of the CardGame class, which handles interrupting all other threads, allowing them to finish off their turn. As they check if the game is over, they should finish up after that turn. One performance issue found is that players will be able to undertake multiple turns until the change in winner is detected due to different threads running at different speeds. Players can also get stuck if it's thread speeds off and empties the deck, causing it to have to wait for another card to be put in by the player to it's left, although it can be argued that this keeps the game fairer between players.

Testing Design

CardGame Tests

CardGame is the main class, which contains the main method, this is very hard to automate testing for as automating tests for user inputs is very fiddly, so we tested this method by running packs which will allow the game to end with an inconsistent winner (meaning the treads are acting at different speeds allowing at least one player to win). Other than the main method, the majority of the non getter or setter methods are tested.

Starting with loadPack, the difficulty we ran into a number of times is that methods should be private within the class for proper coding practices, but this means it is not easy with JUnit 5 tests to generate a test to act on the private method. The simplest way around this was to create a public get_method for the methods we would like to test. The First implementation of this is with get_loadPack.

loadPack takes a file string and a length n of the number of players in the game, to test this method we created a test suite containing 5 different attempts to load a pack. First was to load a valid pack, the next load a pack that is too short for the given number of players, the next a pack with a negative card in and the final one a pack with a zero in it. These tests make sure that the method both works and fails gracefully (returns null) when the pack is invalid. For these tests XPack.txt needs to be present in the file for the method to be able to access it. The final test for loadPack is to check that the code does not crash if the file is not found.

Moving on to the test suite for testing deck and player from the perspective of the CardGame class. The first two functions create an array of players or decks and then checks that those players and decks are present in the CardGame Class. As before the createDecks and createPlayers are private methods for testing purposes so we implemented getter functions for those methods. The test for createPlayers also checks that on creation of the player its attributes are filled but however empty in this case.

Testing for the function dealCardsToPlayers is more complex as it checks that the cards added to the players are found in the hands of the correct players and that the correct number of cards are placed into the correct hands, and finally that the cards are dealt in a round robin fashion to meet the requirement of the specification. This method is again accessed through get_dealCardsToPlayers.

This is almost identical to the dealCardsToDecks as a deck and hand both inherit from cardCollection, so I won't repeat the above, I will however just highlight the testing of the round robin dealing by having only two decks and checking that all even cards are in one deck and all the odd are in the other.

CardCollection Tests

The Tests for card collection are split up into two flavours EmptyCardCollection and NonEmptyCardCollection, we have put these into two test suites, the only thing they share is a setUp method that is applied before each test. For the Empty one it resets the empty card collection instance to make sure that it is empty. Whereas for the non-empty one it generates a new card collection instance and adds a set of cards to it to aid in testing.

The tests for the EmptyCardCollection consist of checking its empty checking the toSting function returns an empty string and checking the iterator returns a false value for the next element in the queue in this instance.

The tests for the NonEmptyCardCollection consists of the same first two tests, but this time the cardCollection contains cards, it then tests the getCard method to check that the iterator is set up as a queue as the first card put into the collection should be returned first. Then the Removecard function is tested to moth check the correct cards are returned with the queue structure.

Card Tests

Card is a simple class that has simple tests. I have split them up into mildly unnecessary test suites but why break type. The first test suite has the test that tests card creation with a correct value, as it should be impossible for a non-positive int to be turned into a card instance thanks to the loadPack method in CardGame. The second test suite checks that the toString method works for a given value on a fresh card. Finally, there are some equality checks to make sure that the values of cards can be the same, but an instance of a card will remain unique and vice versa.

Deck Tests

The deck tests repeat some of the tests previously completed but from a different point of view. The first test suite tests the creation of a deck, simply testing that a deck exists and that the deck is empty. The next test suite tests that the output of the deck when it is called, is correct with what is expected, granted some cards are put into the deck. This tests that the files created by running the card game will contain the correct data. The final test suite tests that after creating two empty decks their identifiers are not the same so they can be differentiated without containing any cards.

The test you may see is missing is to check that removing a card from a deck is done in CardGame, and in Player testing. This is not the only instance of something like this, but this is one of the more obvious ones.

Hand Tests

As I have said before both the hand and deck extend card collection so the tests for both are very similar. This applies to the first test suite that is a carbon copy of the first in deck. However, the first difference about the hand is the method to get a random card from the hand and return it as the card that will be discarded to the left deck.

To test this method, we create an instance of the hand and a set of values as it is easier to test this method if there are no duplicates. Then add cards to the hand with the values selected then from there run the get random card function until a card is returned and check the value of the card is in the original set of values. There is no need to test if the hand is empty as this is not possible.

The final test suite in Hand test is the one to check that remove card works as intended, the getRandomCard method gets a possible card to remove whereas this test just checks that if two cards are created and added to a hand that the size of the hand is what it should be as you remove cards from the hand. We can also check that the correct card is removed by checking that the remaining card is the correct one.

Player Tests

This like the main class of the CardGame contains a run function that is run to allow the multi-threading of the card game, this again adds complication to the testing as you again have to check file output to test.

But to start with the test suite for player creation tests creating a player with two empty decks and then checking that all of the attributes of the player are not null to validate that the instance was created correctly.

The next test suite tests the ability of a player instance to draw and discard a card. It does this by adding 4 cards to the left deck and then adding 4 cards to the hand of the player by using the draw function in the player class, which adds a card to the hand by checking that it is in the left deck and then moving if front the left deck into the hand of the player. From there the hand is then checked to be the correct size and to have the correct top card.

The player then discards a card from the hand using the get random discard function, in doing this it also adds it to the right deck of the player. The final set is to check that the cards are in the correct place and the size of the hand has decreased through the player instance.

The final test suite checks that with a winning initial hand an instance of the player wins and exits correctly and outputs the correct statement to its output file. It does this by setting up a player instance adding 4 cards to it of the value that the player is looking for then adding a card to the left deck and running that player thread and checking the output file for the win line. I separated the file checking into a separate method for code clarity.